

Лабораторная работа №3. Flutter: структура UI и компонентный подход

Цель работы: Научиться строить UI через дерево виджетов, создавать собственные виджеты-классы, разбивать код на файлы и передавать данные через параметры.

Часть 1: Подготовка проекта

Шаг 1. Создание проекта

1. Перейдите в папку с вашими проектами по **Flutter**:

cd Documents/Flutter (как пример)

2. Создайте новый проект:

flutter create flutter_lab3_app

3. Перейдите в папку проекта:

cd flutter_lab3_app

4. Создайте папку для скриншотов:

mkdir img

5. Запустите приложение в Chrome:

flutter run -d chrome

6. Нажмите **q** в терминале для выхода из режима запуска.

Шаг 2: Инициализация Git

1. Откройте приложение в VS Code:

code .

2. Откройте встроенный терминал VS Code: **Ctrl+`** или через меню **Terminal** → **New Terminal**.

3. Инициализируйте репозиторий с основной веткой main:

git init -b main

4. Настройте **.gitignore**. Откройте файл **.gitignore** в корне проекта и добавьте в конец:

```
# Платформы, которые не используем в данной работе
android/
ios/
linux/
macos/
windows/
# IDE и системные файлы
.idea/
*.iml
.DS_Store
Thumbs.db
```

5. Добавьте все файлы в индекс:

git add .

6. Сделайте первый коммит:

```
git commit -m "Initial commit: create Flutter project"
```

Шаг 3. Подключение к GitHub

1. Создайте новый **public**-репозиторий на GitHub:

Flutter_Lab3

Без **README**, **.gitignore** и **License**.

2. Привяжите локальный репозиторий:

```
git remote add origin <URL_вашего_репозитория>
```

```
git push -u origin main
```



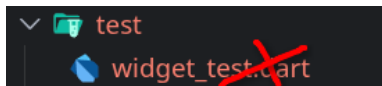
СКРИНШОТ 1: Сделайте скриншот терминала с выводом пуша. Сохраните в `img/step1_ВашаФамилия.png`

Часть 2. Строим приложение: от текста к структуре

Шаг 1. Чистый старт и Hello World

Подготовка файлов

1. Откройте `lib/main.dart` и удалите **весь** код — файл должен стать пустым.
2. Также удалите файл `test/widget_test.dart`



Повторение с прошлой пары

Попробуйте вспомнить и написать самостоятельно конструкцию, которая выводит **Hello world!**. Не подглядывайте — это важно для закрепления.

После того как напишете, сверьтесь с правильным вариантом:

```
import 'package:flutter/material.dart';

Run | Debug | Profile
void main() {
  runApp(MaterialApp(home: (Text("Hello world!"))));
}
```

Запустите приложение любым знакомым способом и убедитесь что текст отображается на экране.

Шаг 2. Виджет Scaffold — каркас экрана

Оберните Text в Scaffold:

```
runApp(MaterialApp(home: (Scaffold(body: Text("Hello world!")))))
```

Шаг 3. Убираем баннер DEBUG

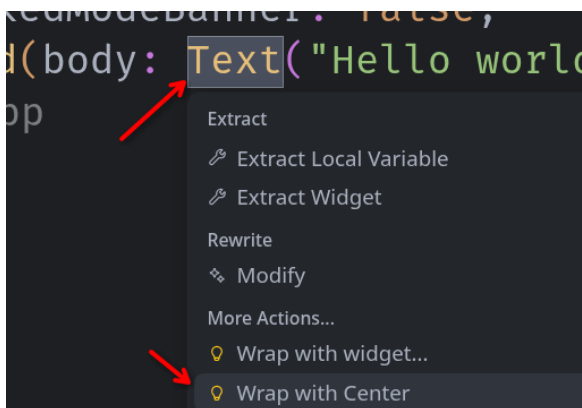
Добавьте параметр `debugShowCheckedModeBanner: false` в `MaterialApp`:

```
MaterialApp(  
  + debugShowCheckedModeBanner: false,  
  home: (Scaffold(  

```

Шаг 4. Градиент фона

1. Оберните `Text` в `Center` через рефакторинг (**Ctrl+Shift+R** на `Text` и выберите **Wrap with Center**).



2. Далее кликните на слово `Center` в коде правой кнопкой мыши.

3. Выберите **Refactor...** (**Ctrl+Shift+R**).

Refactor...

Ctrl+Shift+R

4. В списке выберите **Wrap with Container**.

💡 Wrap with Container

Код станет таким:

```
runApp(  
  MaterialApp(  
    debugShowCheckedModeBanner: false,  
    home: (Scaffold(  
      body: Container(  
        child: Center(  
          child: Text("Hello world!"),  
        ), // Center  
      ), // Container  
    ), // Scaffold  
  ), // MaterialApp  
);
```

Добавляем BoxDecoration и градиент

Параметр **decoration** у **Container** принимает объект **BoxDecoration** — он описывает визуальное оформление контейнера: цвет, градиент, скругление углов, тень, границу.

Добавьте **decoration** с градиентом внутрь **Container**:

```
body: Container(  
  +decoration: BoxDecoration(),  
  child: Center(  

```

Далее допишем весь код:

```
debugShowCheckedModeBanner: false,  
home: (Scaffold(  
  body: Container(  
    + decoration: BoxDecoration(  
      gradient: LinearGradient(  
        colors: [  
          Colors.white,  
          Colors.blue,  
          Colors.red,  
        ],  
      ), // LinearGradient  
    ), // BoxDecoration  
    child: Center(  
      child: Text("Hello world!"),  
    ), // Center  
  ), // Container
```

Шаг 5. Направление градиента

Добавьте два параметра в **LinearGradient** после списка **colors**:

```
    colors: [  
      Colors.white,  
      Colors.blue,  
      Colors.red,  
    ],  
    + begin: Alignment.topCenter,  
    end: Alignment.bottomCenter,  
  ), // LinearGradient
```

Шаг 6. Стилизация текста

Обновите виджет **Text**:

```
child: Center(
  child: Text(
    "Hello world!",
    style: TextStyle(
      color: Colors.white,
      fontSize: 32,
    ), // TextStyle
  ), // Text
), // Center
```

 **СКРИНШОТ 2:** Скриншот готового кода в `main.dart` с крупным белым текстом на градиентном фоне. Сохраните в `img/step2_ВашаФамилия.png`

Сделайте коммит:

git add .

git commit -m "Create basic UI with gradient background and styled text"

Часть 3. Строим свой виджет

Посмотрите на дерево виджетов которое у нас сейчас есть:

MaterialApp

└─ **Scaffold**

└─ **Container**

└─ **Center**

└─ **Text**

Оно уже достаточно глубокое, и дальше будет только расти. В реальных Flutter-приложениях деревья виджетов могут быть огромными — и держать всё в одном методе `main()` быстро становится неудобно.

Правильный подход — **разбивать UI на отдельные классы-виджеты** и переиспользовать их. Это то же самое что выносить логику в отдельные функции или классы в Kotlin и C# — код становится читаемее, проще в поддержке и легко переиспользуется в других частях приложения.

Вынесем наш **Container** со всем содержимым в отдельный класс.

Шаг 1. Создаём класс виджета

Классы в Dart объявляются так же как в Kotlin и C#. Единственное отличие — наследование - вместо `:` используется ключевое слово **extends**.

Создайте новый класс **за пределами** метода `main()`, в конце файла:

```
class GradientContainer extends StatelessWidget {  
  
}
```

Название **GradientContainer** отражает что делает этот виджет — отображает контейнер с градиентом. Называйте свои виджеты по их назначению.

StatelessWidget — базовый класс Flutter для виджетов без состояния. Наследуясь от него, мы получаем всю инфраструктуру Flutter для отрисовки виджета на экране. Это аналог того, как в Jetpack Compose **MainActivity** наследовалась от **ComponentActivity** чтобы получить доступ к возможностям фреймворка.

Сразу появится ошибка — VS Code подчёркивает название класса. Наведите курсор:

```
Missing concrete implementation of 'StatelessWidget.build'.  
Try implementing the missing method, or make the class  
abstract. dart(non_abstract_class_inherits_abstract_member)
```

StatelessWidget требует реализовать метод **build()** — именно он отвечает за то, что виджет отрисовывает на экране. Без него класс неполный.

Шаг 2. Добавляем метод **build()**

Добавим метод внутрь класса постепенно, разбирая каждое добавление.

1. Добавьте пустой метод:

```
class GradientContainer extends StatelessWidget {  
  + build() {}  
}
```

Одна ошибка пропала, но появилась другая:

```
The body might complete normally, causing 'null' to  
be returned, but the return type, 'Widget', is a  
potentially non-nullable type.  
Try adding either a return or a throw statement at  
the end. dart(body_might_complete_normally)
```

Метод **build()** должен возвращать виджет — а сейчас он ничего не возвращает.

2. Добавьте возвращаемый тип **Widget**:

```
Widget build() {}
```

3. Метод должен переопределять родительский и принимать параметр **context**. Добавьте **@override** и аргумент:

```
@override  
Widget build(BuildContext context) {}
```

BuildContext — это объект, который содержит информацию о положении виджета в дереве. Flutter передаёт его автоматически при каждой перерисовке. Пока он нам не нужен напрямую, но метод без него работать не будет.

@override вы уже знаете из Kotlin и C# — он сигнализирует что мы переопределяем метод родительского класса. Аннотация **@override** не обязательна для компиляции, но является хорошей практикой, потому что позволяет компилятору проверить, что мы действительно переопределяем метод родительского класса.

Шаг 3. Переносим виджеты в **build()**

Теперь добавьте **return** внутрь метода и перенесите **Container** из **main()** — вырежьте его и вставьте после **return**:

Метод **main()**:

```
void main() {  
  runApp(  
    MaterialApp(  
      debugShowCheckedModeBanner: false,  
      home: (Scaffold(  
        body: GradientContainer(),  
      )), // Scaffold  
    ), // MaterialApp  
  );  
}
```

Класс **GradientContainer**:

```

class GradientContainer extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Container(
      decoration: BoxDecoration(
        gradient: LinearGradient(
          colors: [
            Colors.white,
            Colors.blue,
            Colors.red,
          ],
          begin: Alignment.topCenter,
          end: Alignment.bottomCenter,
        ), // LinearGradient
      ), // BoxDecoration
      child: Center(
        child: Text(
          "Hello world!",
          style: TextStyle(
            color: Colors.white,
            fontSize: 32,
          ), // TextStyle
        ), // Text
      ), // Center
    ); // Container
  }
}

```

Сохраните и убедитесь что приложение работает так же как раньше — визуально ничего не изменилось, но код теперь разбит на логические части.

Шаг 4. Добавляем конструктор

Осталось одно предупреждение — VS Code выделяет название класса жёлтым:

```

Constructors for public widgets should have a named 'key'
parameter.
Try adding a named parameter to the
constructor. dart(use\_key\_in\_widget\_constructors)

```

Flutter рекомендует добавить конструктор с параметром `key`. Добавьте его первой строкой внутри класса:

```

class GradientContainer extends StatelessWidget {
  const GradientContainer({super.key});

```

- **const** — конструктор константный: если все параметры известны на этапе компиляции, Flutter может оптимизировать создание виджета и не пересоздавать его без необходимости.

- **super.key** — передаёт параметр `key` в конструктор родительского класса `StatelessWidget`. `key` используется Flutter для идентификации виджетов в дереве при перестройке UI — это важно для оптимизации и корректной работы анимаций.

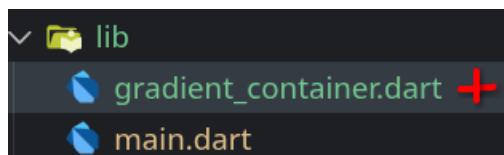
Предупреждение исчезнет.

Шаг 5. Выносим виджет в отдельный файл

Сейчас оба класса — **main()** и **GradientContainer** — живут в одном файле **main.dart**. Для небольшого примера это допустимо, но в реальных проектах каждый виджет принято хранить в отдельном файле. Это стандартная практика Flutter-разработки: легче ориентироваться в проекте, проще искать нужный виджет и переиспользовать его в других местах.

Создайте новый файл

В папке **lib/** создайте файл **gradient_container.dart**

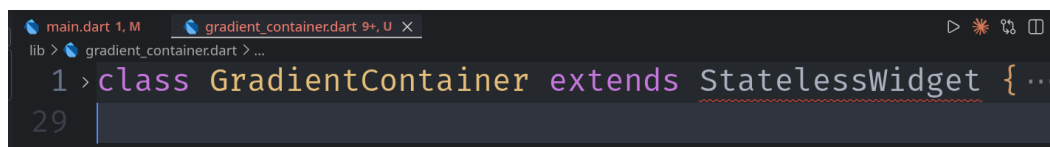


💡 Обратите внимание на соглашение об именовании: файлы в Dart называются в стиле **snake_case** — слова в нижнем регистре через нижнее подчёркивание. Это стандарт Dart, которого принято придерживаться.

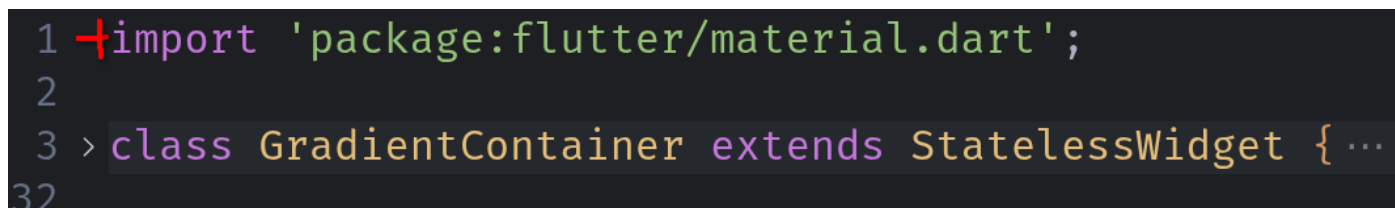
Перенесите класс

Вырежьте весь класс `GradientContainer` из `main.dart` и вставьте его в новый файл `gradient_container.dart`.

Сразу появится ошибка — Flutter больше не знает что такое `Colors`, `Container`, `LinearGradient` и другие виджеты, потому что импорт остался в `main.dart`:

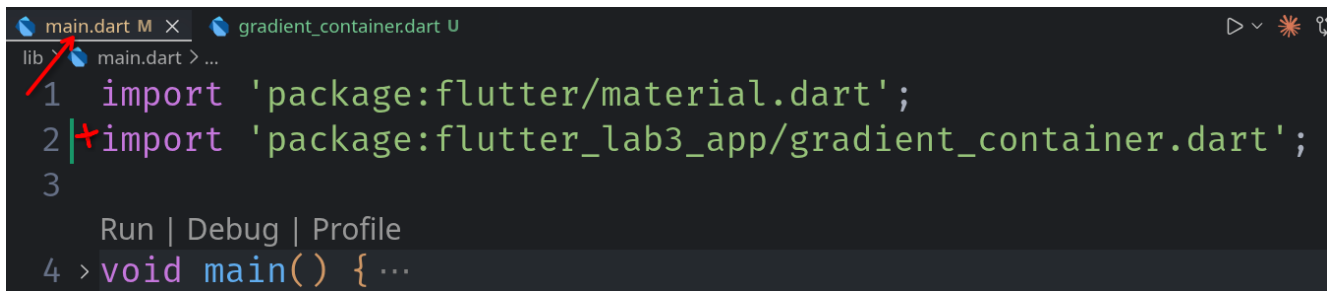


Добавьте его в начало нового файла **gradient_container.dart**:



Добавьте импорт в `main.dart`

Теперь `main.dart` не знает о существовании `GradientContainer` — класс переехал в другой файл. Добавьте импорт в начало `main.dart`:



```
main.dart M x gradient_container.dart U
lib > main.dart > ...
1 import 'package:flutter/material.dart';
2 import 'package:flutter_lab3_app/gradient_container.dart';
3
Run | Debug | Profile
4 > void main() { ...
```

После этого все ошибки исчезнут.

Запустите приложение и убедитесь что всё работает так же, как до разбивки на файлы.

 **СКРИНШОТ 3:** Скриншот VS Code с готовым классом `GradientContainer`.

Сохраните в `img/step3_ВашаФамилия.png`

Сделайте коммит:

`git add .`

`git commit -m "Extract GradientContainer to a separate file"`

Шаг 6. Практика создания виджета

Вы уже умеете выносить виджеты в отдельные классы и файлы. Теперь закрепим это самостоятельно.

Задание: Вынос текста в отдельный виджет

Что нужно сделать

1. Создайте новый файл `styled_text.dart` в той же папке, где находится `main.dart`
2. В файле `styled_text.dart` создайте класс `StyledText`:
 - Класс должен быть наследником `StatelessWidget`
 - Конструктор должен принимать параметр `String text`
 - Виджет должен возвращать `Text` с такими же стилями: белый цвет, размер 32
 - Не забудьте про ключ (`key`) в конструкторе
3. В исходном файле с `GradientContainer`:
 - Удалите `Text` виджет из `child`: `Center(...)`

- Вместо него используйте новый виджет `StyledText` с текстом "Hello world!"
- Добавьте необходимый импорт

Подсказки по структуре

Файл `styled_text.dart` должен содержать:

- Импорт `package:flutter/material.dart`
- Класс с конструктором, принимающим текст
- Поле для хранения текста (какой модификатор использовать?)
- Метод `build`, возвращающий `Text` с переданным текстом и стилями

Файл с `GradientContainer` должен:

- Импортировать `styled_text.dart`
- Использовать `<StyledText>('Hello world!')` вместо старого `Text`

Проверьте себя

После выполнения код должен работать точно так же, как и раньше, но теперь текст вынесен в отдельный виджет. **Если не справитесь в конце работы будет решение.**



СКРИНШОТ 4: Скриншот VS Code с тремя файлами в папке `lib/` и кодом `StyledText` в `styled_text.dart`. Сохраните в `img/step4_ВашаФамилия.png`

Сделайте коммит:

`git add .`

`git commit -m "Extract StyledText into a separate widget and file"`

Шаг 7. Переменные на уровне файла

Переменные в Dart можно объявлять не только внутри класса или метода, но и на **уровне файла** — вне любого класса. Это удобно для значений, которые используются в классе, но логически не являются его частью.

Вынесем направление градиента в две переменные. Откройте файл `gradient_container.dart` и объявите их **перед** классом:

```
const startAlignment = Alignment.topCenter;
const endAlignment = Alignment.bottomCenter;
```

Измените направление градиента **внутри** класса `GradientContainer`:

```
begin: startAlignment,
end: endAlignment,
```

Теперь если нужно изменить направление градиента — меняете одно место в начале файла, а не ищите значения внутри класса.

Шаг 8. Делаем виджет гибким — передаём данные через параметры

Сейчас **StyledText** всегда показывает одно и то же: **"Hello world!"**. Текст жёстко зашит внутри виджета — такой виджет нельзя переиспользовать.

Хороший виджет **получает данные снаружи**, а не хранит их внутри. Сделаем **StyledText** гибким — пусть принимает текст как параметр.

Откройте файл **styled_text.dart**.

Добавляем поле класса

Добавьте **поле text** **внутри класса** перед конструктором:

```
class StyledText extends StatelessWidget {  
  + final String text;
```

- **final** — переменная, которая присваивается один раз и больше не меняется. Аналог **val** в Kotlin и **readonly** в C#. Для данных виджета это правильный выбор: **StatelessWidget** не меняет своё состояние после создания.
- **text** — это поле класса, аналог **property** в C#.

Сразу появится ошибка — конструктор не инициализирует новое поле:

```
StyledText({super.key});  
All final variables must be initialized, but 'text' isn't.  
Try adding an initializer for the field. dart(final_not_initialized_constructor)
```

Обновляем конструктор

Добавьте **text** в конструктор:

```
const StyledText(this.text, {super.key});  
// Override
```

- **this.text** — сокращённый синтаксис Dart: параметр конструктора сразу присваивается полю класса. В C# пришлось бы писать **this.text = text** отдельно.

Используем поле в build()

Замените строку **"Hello world!"** на поле **text**:

```
@override  
Widget build(context) {  
  return Text(  
    text,
```

Обновляем вызов в GradientContainer

Откройте файл `gradient_container.dart` и передайте текст при вызове виджета:

```
child: Center(  
  child: StyledText("Hello world!"),  
), // Center
```

Теперь **StyledText** можно использовать в любом месте приложения с любым текстом — просто передав нужную строку при вызове.

Сделайте коммит:

git add .

git commit -m "Make StyledText reusable by accepting text as a parameter"

Шаг 9. Практика: передача данных через параметры

Вы уже умеете передавать параметры виджетам. Теперь закрепим самостоятельно.

Задание: сделайте виджет **GradientContainer** гибким — вынесите список цветов градиента **colors** в параметр (**color1**, **color2** и **color3**), который передаётся снаружи при вызове виджета.

В итоге вызов в **main.dart** должен выглядеть примерно так:

```
home: (Scaffold(  
  body: GradientContainer(  
    Colors.white,  
    Colors.blue,  
    Colors.red,  
  ), // GradientContainer
```

💡 Вспомните шаг 8: добавить поле класса с **final**, обновить конструктор, использовать поле внутри **build()**. **Если не справитесь в конце работы будет решение.**

📸 **СКРИНШОТ 5:** Скриншот VS Code с обновлённым **GradientContainer**, принимающим **colors** как параметр. Сохраните в **img/step5_ВашаФамилия.png**

Сделайте коммит:

git add .

git commit -m "Make GradientContainer reusable by accepting colors as a parameter"

Часть 4. Создаём приложение ROLL DICE APP

Теперь мы переделаем наш проект — уберём текст и сделаем игру с кубиком.

GradientContainer останется фоном, а содержимое полностью изменится

При нажатии на кнопку приложение будет показывать случайную грань кости.

В процессе мы изучим:

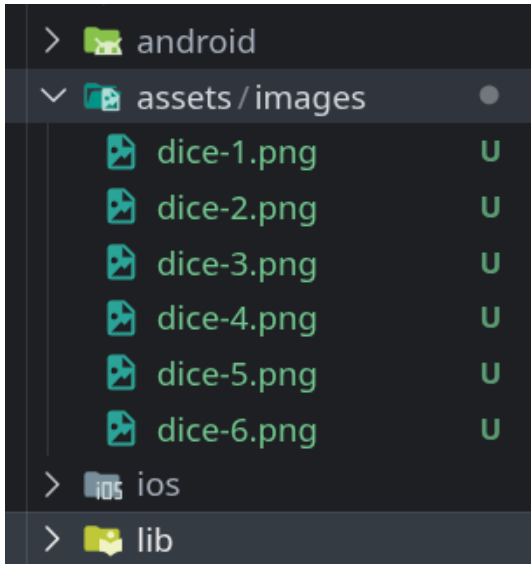
- работу с **assets** (ресурсами приложения)
- добавление **изображений**
- создание **кнопки**
- управление **состоянием интерфейса**
- использование **StatefulWidget**
- генерацию **случайных чисел**

Шаг 1. Добавление изображения

К лабораторной работе прикреплён архив с изображениями игровых костей.

1. В корне проекта создайте папку **assets**

2. Внутри этой папки создайте папку **images/** и скопируйте **в папку images** все изображения из архива:



3. Подключите assets в pubspec.yaml

В файле **pubspec.yaml** в секцию **flutter** добавить **assets** :

```
flutter:  
  uses-material-design: true  
  assets:  
    - assets/images/
```

Обратите внимание на отступы — *YAML* чувствителен к ним.

Если указать папку со слешем на конце, *Flutter* автоматически подключит все файлы внутри неё.

4. Покажем изображение в интерфейсе

1. Откройте файл **gradient_container.dart**

2. И замените вызов дочернего элемента **StyledText** на виджет изображения:

```
child: Center(  
  child: Image.asset(  
    'assets/images/dice-1.png',  
  ), // Image.asset  
) // Center  
); // Container
```

Правильно пропишите путь до изображения

5. Запустите приложение и проверьте что картинка появилась.

6. Добавьте параметр высоты:

```
child: Image.asset(  
  'assets/images/dice-1.png',  
  +width: 300,  
) // Image.asset
```

Сделайте коммит:

git add .

git commit -m "Add dice images and display first dice"

Шаг 2. Добавление кнопки

Продолжаем работу в файле **gradient_container.dart**

Сейчас в **Center** находится только изображение. Но нам нужно разместить картинку и кнопку вертикально. Для этого используем виджет **Column**.

1. Оборните Image в Column

Щёлкните правой кнопкой по **Image** и выберите **Refactor** → **Wrap with Column**

💡 Wrap with Column

2. Добавьте кнопку

В тот же список **children** добавьте виджет **TextButton**, после нашего изображения:

```

child: Center(
  child: Column(
    children: [
      Image.asset(
        'assets/images/dice-1.png',
        width: 300,
      ), // Image.asset
      TextButton(
        onPressed: onPressed,
        child: child,
      ), // TextButton
    ],
  ), // Column
), // Center

```

3. Настройка кнопки:

- В параметр `child` поместите виджет `Text` с надписью "Roll Dice"

```

TextButton(
  onPressed: onPressed,
  child: Text("Roll Dice"),
)

```

- Для параметра `onPressed` создайте отдельный метод `void rollDice() {}` внутри класса.

```

void rollDice() {} +

@override
Widget build(BuildContext context) {

```

- Передайте имя этого метода в `onPressed` без скобок (как ссылку на функцию):

`onPressed: rollDice.`

```

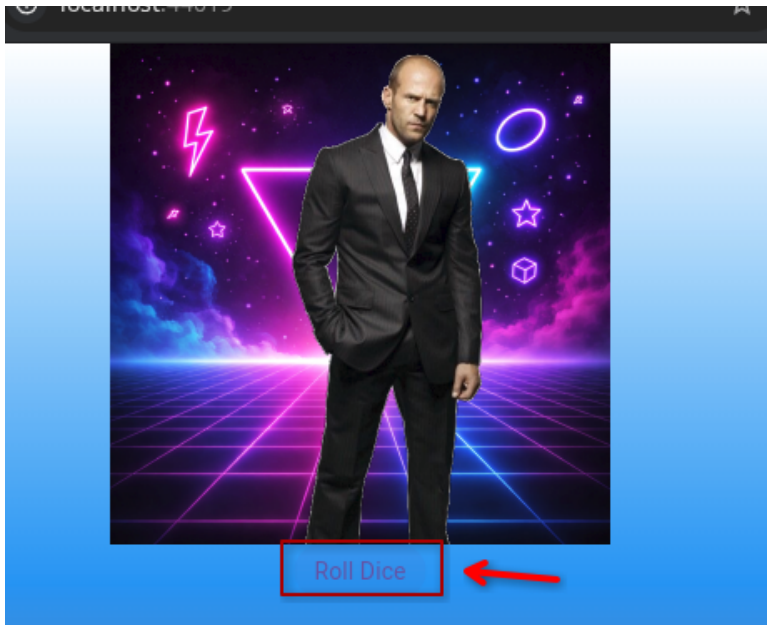
TextButton(
  onPressed: rollDice,
  child: Text("Roll Dice")
)

```

Другой подход заключается в передаче пустой анонимной функции, но так мы не сможем менять логику кнопки позже (**не нужно переписывать**):


```
onPressed: () {},
```

4. Запустите ваше приложение, и проверьте что у вас появилась кнопка:



Сделайте коммит:

```
git add .
```

```
git commit -m "Add Roll Dice button"
```

Шаг 3. Стилизация и отступы

1. В виджете **Column** добавьте параметр **mainAxisSize: MainAxisSize.min**. По умолчанию колонка занимает всё вертикальное пространство, а это значение позволит ей центрироваться внутри виджета **Center**:

```
child: Center(  
  child: Column(  
    +mainAxisSize: MainAxisSize.min,  
    children: [  
      
```

2. В виджет **TextButton** добавьте аргумент **style**. Используйте метод **TextButton.styleFrom** для настройки:

- `foregroundColor: Colors.white` — изменит цвет текста на белый.
- `textStyle: TextStyle(fontSize: 28)` — увеличит размер шрифта.

```

TextButton(
  onPressed: rollDice,
  style: TextButton.styleFrom(
    foregroundColor: Colors.lime,
    textStyle: const TextStyle(
      +   fontSize: 30,
    ), // TextStyle
  ),
  child: Text("Roll Dice"),
), // TextButton

```

3. Отступы (на выбор): Чтобы добавить пространство между изображением и кнопкой, используйте один из способов:

Вариант А: В style кнопки добавьте padding:

```

style: TextButton.styleFrom(
  padding: const EdgeInsets.only(
    +   top: 20,
  ), // EdgeInsets.only
  foregroundColor: Colors.lime,
)

```

Вариант Б: В список children виджета Column (между изображением и кнопкой) добавьте SizedBox(height: 20).

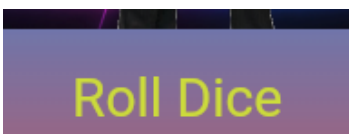
```

Image.asset( // Image.asset ...
+ const SizedBox(height: 20),
TextButton( // TextButton ...

```

Оставьте один из вариантов, другой закомментируйте.

4. Сохраните файлы и убедитесь, что кнопка отображается по центру, имеет лаймовый крупный текст и необходимый отступ сверху.



Сделайте коммит:

git add .

git commit -m "Style roll button and adjust layout"

Шаг 4. Логика смены изображения

1. В классе **GradientContainer** объявите изменяемую переменную `activeDiceImage` и инициализируйте её путём к изображению:

```
var activeDiceImage =  
    'assets/images/dice-1.png';
```

2. Уберите ключевое слово **const** у виджета **GradientContainer**, так как внутри появились изменяемые данные:

```
class GradientLinear exten  
    const GradientLinear(  
        ...  
    )
```

3. В виджете **Image.asset** замените жёстко заданный путь на переменную `activeDiceImage`:

```
Image.asset(  
    activeDiceImage,  
    width: 300,  
), // Image.asset
```

4. В методе **rollDice** присвойте переменной `activeDiceImage` новый путь (например, 'assets/images/dice-4.png'):

```
void rollDice() {  
    +activeDiceImage = 'assets/images/dice-4.png';  
}
```

5. Запустите приложение и попробуйте нажать на кнопку. Ничего не происходит.

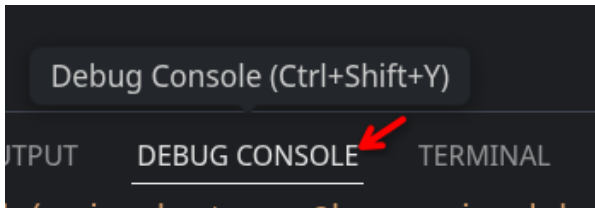
6. Возможно наш метод не выполняется. Давайте добавим проверку. Добавьте в метод `rollDice` вывод, чтобы убедиться через консоль отладки, что функция вызывается:

```
void rollDice() {  
    activeDiceImage = 'assets/images/dice-4.png';  
    +print('Изменили картинку');  
}
```

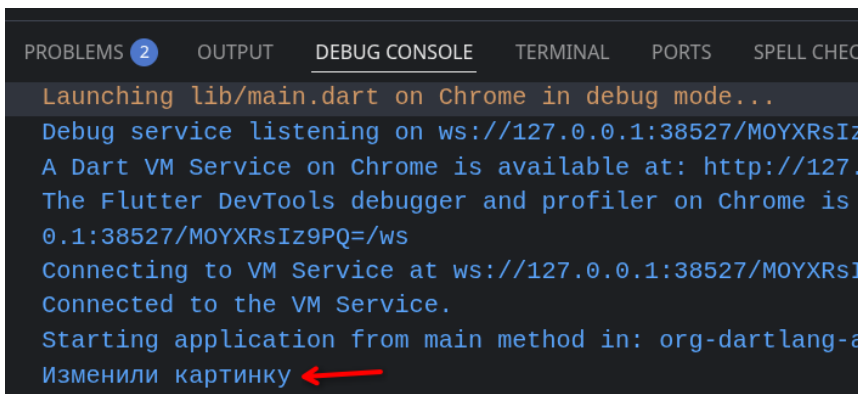
7. Также у нас с вами в `main.dart` появилась ошибка. Нужно убрать `const` у `MaterialApp`:

```
void main() {
  runApp(
    const MaterialApp(
```

8. Перезапустите приложение. Откройте Debug Console



Попробуйте нажать на кнопку, и увидите что вывод появился:



После сохранения переменная будет меняться, но изображение на экране пока не обновится визуально (для этого потребуется дополнительный механизм в следующем шаге).

Сделайте коммит:

git add .

git commit -m "Add dice image state and rollDice method"

Шаг 5. StatefulWidget и управление состоянием

Данная проблема заключается в том, что мы используем виджет StatelessWidget, который не сохраняет состояние.

Почему StatelessWidget больше не хватает

До сих пор все наши виджеты были StatelessWidget — они отрисовываются один раз и не меняются. Это отлично работает для статичного UI: текст, картинки, фон.

Но наше приложение должно реагировать на нажатие кнопки и менять картинки. Виджет должен помнить своё текущее состояние и уметь его менять.

Для этого во **Flutter** есть **StatefulWidget**.

Сравним два подхода:

	StatelessWidget	StatefulWidget
Состояние	Нет	Есть — хранится в объекте State
Перерисовка	Только при пересоздании	По вызову setState()
Когда использовать	Статичный UI	UI который меняется по действию пользователя
Аналог в Compose	Composable без state	remember { mutableStateOf() }

StatefulWidget состоит из двух классов:

- сам виджет — описывает конфигурацию
- объект State — хранит состояние и содержит метод build()

Создаём виджет

1. Создайте в папке **lib/** новый файл **dice_roller.dart**
2. Откройте его и напишем виджет:

```
import 'package:flutter/material.dart';

class DiceRoller extends StatefulWidget {
  const DiceRoller({super.key});

  @override
  State<StatefulWidget> createState() {
    return _DiceRollerState();
  }
}

class _DiceRollerState extends State<DiceRoller> {
  @override
  Widget build(BuildContext context) {
    return "сюда вставим виджет";
  }
}
```

Как работает StatefulWidget

StatefulWidget состоит из двух частей:

1. Класс виджета (DiceRoller)

Он является неизменяемой конфигурацией.

2. Класс состояния (_DiceRollerState)

Он хранит изменяемые данные и содержит метод build().

Когда состояние изменяется, **Flutter** вызывает метод **setState()**, после чего виджет перестраивается.

Имя **_DiceRollerState** начинается с **_** - это **private** класс в **Dart**

3. Далее нам нужно **вырезать** виджет **Column** из файла **gradient_container.dart** внутрь метода **build** класса **_DiceRollerState**:

```
class _DiceRollerState extends State<DiceRoller> {
  @override
  Widget build(BuildContext context) {
    return Column(
      mainAxisAlignment: MainAxisAlignment.min,
      children: [
        Image.asset(activeDiceImage, width: 300),
        const SizedBox(height: 20),
        TextButton(
          onPressed: rollDice,
          style: TextButton.styleFrom(
            // padding: const EdgeInsets.only(
            //   top: 20,
            // ),
            foregroundColor: Colors.lime,
            textStyle: const TextStyle(fontSize: 30),
          ),
          child: Text("Roll Dice"),
        ), // TextButton
      ],
    ); // Column
  }
}
```

4. Переместите переменную **activeDiceImage** и метод **rollDice** из **GradientContainer** в класс **_DiceRollerState**:

```
var activeDiceImage = 'assets/images/dice-1.png';

void rollDice() {
  activeDiceImage = 'assets/images/dice-4.png';
  print('Изменили картинку');
}
```

Вырезаем

```
class _DiceRollerState extends State<DiceRoller> {
  var activeDiceImage = 'assets/images/dice-1.png';

  void rollDice() {
    activeDiceImage = 'assets/images/dice-4.png';
    print('Изменили картинку');
  }
}
```

Вставляем

```
@override
Widget build(BuildContext context) {
```

5. Убедитесь, что **GradientContainer** снова может быть помечен как **const**, так как он больше не хранит изменяемое состояние.

То же самое сделаем в **main.dart** для **MaterialApp**:

```
void main() {  
  runApp(  
    const MaterialApp(  

```

Это хорошая практика — **const** означает что виджет не изменится, и **Flutter** не будет его пересоздавать без необходимости.

6. В **GradientContainer** замените старую разметку на виджет **DiceRoller()**

```
), // BoxDecoration  
child: Center(  
  child: DiceRoller()  
), // Center  
); // Container
```

7. Добавьте импорт файла **dice_roller.dart**

```
import 'package:flutter_lab_3_app/dice_roller.dart';
```

8. В классе **_DiceRollerState** в методе **rollDice** оберните изменение переменной в **setState()** { **activeDiceImage** = ...; })

```
void rollDice() {  
  [setState(() {  
    activeDiceImage = 'assets/images/dice-4.png';  
  }]);  
  print('Изменили картинку');  
}
```

Это сообщит Flutter о необходимости перерисовки экрана. Теперь при нажатии на картинку, у нас изменилось изображение. Также можете удалить строку **print()** — она была нужна для диагностики, в финальном коде она не нужна.

Сделайте коммит:

git add .

git commit -m "Refactor dice logic into StatefulWidget DiceRoller"

Шаг 6. Генерация случайного числа

Сейчас при нажатии кнопки всегда показывается одна и та же кость. Добавим случайность.

1. В начале файла **dice_roller.dart** добавьте импорт для работы с методом **Random**

```
import 'dart:math';
```

2. Вне класса создайте глобальный объект:

```
final randomizer = Random();
```

Создание **Random** внутри функции при каждом нажатии кнопки будет создавать новый генератор случайных чисел. Это не критично, но менее эффективно.

Поэтому создадим один объект **Random** на уровне файла и будем использовать его повторно.

3. В классе **_DiceRollerState** замените переменную пути (**activeDiceImage**) на переменную числа (удалите строку `var activeDiceImage = ...` — она больше не нужна):

```
class _DiceRollerState extends State<DiceRoller> {  
+ var currentDiceRoll = 2;  
  var activeDiceImage = 'assets/images/dice-1.png';  
}
```

4. В методе **rollDice** внутри **setState** обновите значение:

```
void rollDice() {  
  setState(() {  
+ currentDiceRoll = randomizer.nextInt(6) + 1;  
    activeDiceImage = 'assets/images/dice-4.png';  
  });  
}
```

Генерирует число от 1 до 6.

5. В виджете **Image.asset** используйте интерполяцию строк для динамического пути:

```
children: [  
  Image.asset(  
    'assets/images/dice-$currentDiceRoll.png',  
    width: 300,  
  ), // Image.asset
```

Запустите и проверьте работу приложения.



СКРИНШОТ 6: Финальная версия приложения Roll Dice. Кнопка "Roll Dice" и случайная игральная кость на градиентном фоне. Сохраните в **img/step6_ВашаФамилия.png**

Сделайте коммит:

git add .

git commit -m "Add random dice roll logic"

Самостоятельное задание. Оформление README.md

Оформите файла README.md. Напишите:

- Название лабораторной и номер
- Фамилия, имя, группа, дата сдачи
- Что изучили (3–5 пунктов своими словами)
- Скриншот финального приложения
- Ссылка на репозиторий
- **Инструкция по запуску**
- **Ответь на вопросы:**
 1. Зачем выносить виджеты в отдельные файлы? Что изменится если держать всё в main.dart?
 2. Что такое BuildContext? Почему метод build() принимает его как параметр?
 3. Чем StatelessWidget отличается от StatefulWidget? Приведите пример когда нужен каждый из них.
 4. Почему Random() создаётся на уровне файла, а не внутри rollDice()?

Сделайте коммит:

git add .

git commit -m "added README description and completed labs"

git push

Решение задания №1

```
lib > styled_text.dart M × main.dart gradient_container.dart M
1 import 'package:flutter/material.dart';
2
3 class StyledText extends StatelessWidget {
4   const StyledText({super.key});
5   @override
6   Widget build(BuildContext context) {
7     return Text(
8       "Hello world!",
9       style: TextStyle(
10        color: Colors.white,
11        fontSize: 32,
12      ), // TextStyle
13    ); // Text
14  }
15 }

import 'package:flutter/material.dart';
import 'package:flutter_lab3_app/styled_text.dart';

class GradientContainer extends StatelessWidget {
  const GradientContainer({super.key});
  @override
  Widget build(BuildContext context) {
    return Container(
      decoration: BoxDecoration(
        gradient: LinearGradient(
          colors: [
            Colors.white,
            Colors.blue,
            Colors.red,
          ],
          begin: Alignment.topCenter,
          end: Alignment.bottomCenter,
        ), // LinearGradient
      ), // BoxDecoration
      child: Center(
        child: StyledText(),
      ), // Center
    ); // Container
  }
}
```

Решение задания №2

```
class GradientContainer extends StatelessWidget {  
+  [final Color color1;  
  final Color color2;  
  final Color color3;  
  
  const GradientContainer(  
    [this.color1,  
    this.color2,  
    this.color3, {  
      super.key,  
    }]);  
  
  @override  
  Widget build(BuildContext context) {  
    return Container(  
      decoration: BoxDecoration(  
        gradient: LinearGradient(  
          ✓ colors: [color1, color2, color3],
```